

C# プログラミング確認テスト

解答・解説スライド（教員用 / 解説会 投影用）

大問1-1

配点 2点

```
int a = 5;  
int b = 2;  
Console.WriteLine(a + b * 3);
```

正解

11

解説

演算子の優先順位がポイント。掛け算は足し算より先に計算されるため、 $a + b * 3$ は $5 + (2 * 3) = 5 + 6 = 11$ となる。

大問1-2

配点 2点

```
int score = 72;
if (score >= 80)
{
    Console.WriteLine("A");
}
else if (score >= 60)
{
    Console.WriteLine("B");
}
else
{
    Console.WriteLine("C");
}
```

正解

B

解説

if / else if / else は上から順に条件を評価し、最初に成立した条件だけが実行される。score(72)は80未満・60以上なので "B" が出力される。

大問1-3

配点 2点

```
int sum = 0;
for (int i = 1; i <= 5; i++)
{
    sum += i;
}
Console.WriteLine(sum);
```

正解
15

解説

i を1から5まで変化させながら sum に加算する。 $1+2+3+4+5 = 15$ 。継続条件が $i \leq 5$ なので $i = 5$ のときも実行される点に注意。

大問1-4

配点 2点

```
int[] nums = { 3, 6, 9 };  
for (int i = 0; i < nums.Length; i++)  
{  
    Console.Write(nums[i] + " ");  
}
```

正解

3 6 9 （末尾に空白、改行なし）

解説

for ループで添字 i を0からLength-1まで動かし、`nums[i]`を順に出力する。`Console.Write` は `WriteLine` と違い出力後に改行しない。

大問1-5

配点 2点

```
int i = 1;
int product = 1;
while (i <= 4)
{
    product *= i;
    i++;
}
Console.WriteLine(product);
```

正解
24

解説

while (i <= 4) の間、product に i を掛け続ける。 $1 \times 1 \times 2 \times 3 \times 4 = 24$ (4の階乗)。i が5になった時点でループが終わる。

大問1-6

配点 2点

```
int age = 20;
bool hasTicket = true;
if (age >= 18 && hasTicket)
{
    Console.WriteLine("入場OK");
}
else
{
    Console.WriteLine("入場NG");
}
```

正解
入場OK

解説

&& は「両方の条件が真のときだけ真」になる論理演算子。age(20)>=18 も hasTicket(true) も真なので、if の中が実行される。

大問1-7

配点 2点

```
string[] names = { "Yui", "Sora", "Rin" };  
int index = 1;  
Console.WriteLine("こんにちは、" + names[index] + "さん");
```

正解

こんにちは、Soraさん

解説

配列の添字は0から始まるため、names[1] は2番目の要素 "Sora"。文字列の + 演算子は連結（つなげる）として働く。

大問1-8

配点 2点

```
int[] nums = { 1, 2, 3 };  
for (int i = 0; i < nums.Length; i++)  
{  
    nums[i] = nums[i] * nums[i];  
}  
for (int i = 0; i < nums.Length; i++)  
{  
    Console.Write(nums[i] + " ");  
}
```

正解

1 4 9 （末尾に空白、改行なし）

解説

1つ目の for ループで各要素を自分自身と掛けて2乗にする（1→1, 2→4, 3→9）。2つ目の for ループでその結果を順に出力する。

2

コンパイルエラーを探して直す

配点20点（各5点：行番号2点／正しい行の書き直し3点）

大問2-1

配点 5点

```
1: int x = 10  
2: int y = 20;  
3: Console.WriteLine(x + y);
```

正解

行番号：1行目

正しい行：int x = 10;

解説

C#では文の終わりに必ずセミコロン(;)が必要。1行目にセミコロンがないため CS1002 エラーになる。

大問2-2

配点 5点

```
1: int count = 0;  
2: string numberText = "100";  
3: count = numberText;  
4: Console.WriteLine(count);
```

正解

行番号：3行目

正しい行：`count = int.Parse(numberText);`

解説

int型の変数にstring型の値を直接代入することはできない（CS0029）。numberText の中身が "100" という数字に見えても、string型である限り自動変換はされない。int.Parse で明示的に変換する必要がある。

大問2-3

配点 5点

```
1: static void Main(string[] args)
2: {
3:     int num = 10;
4:     if (num = 5)
5:     {
6:         Console.WriteLine("Five");
7:     }
8: }
```

正解

行番号：4行目

正しい行：if (num == 5)

解説

= は代入演算子、== は比較演算子。if の中は本来 bool（真偽値）でなければならないが、num = 5 は「5を代入した結果（int型の5）」になるため、int を bool に変換できずコンパイルエラーになる。

大問2-4

配点 5点

```
1: static void Main(string[] args)
2: {
3:     int result = Add(5, 3;
4:     Console.WriteLine(result);
5: }
6:
7: static int Add(int a, int b)
8: {
9:     return a + b;
10: }
```

正解

行番号：3行目

正しい行：int result = Add(5, 3);

解説

関数を呼び出す Add(5, 3 の括弧が閉じられていない。丸括弧の対応が崩れているため CS1026 などのエラーになる。

3

実行時エラーを探して説明する

配点24点（各6点：発生行1点／理由2点／改善行1点／書き直し2点）

大問3-1

配点 6点

```
1: int[] scores = { 80, 90, 70 };  
2: for (int i = 0; i <= scores.Length; i++)  
3: {  
4:     Console.WriteLine(scores[i]);  
5: }
```

正解

発生行：4行目（i=3のとき）

理由：scores.Lengthは3。有効な添字は0～2だが、
i<=scores.Lengthでi=3でも実行されるため

改善行：2行目

書き直し：for (int i = 0; i < scores.Length; i++)

解説

配列の要素数と、使える添字の範囲（0～要素数-1）がずれるのが典型的な原因。境界値のミス（<=とその意味）を確認する良問。

大問3-2

配点 6点

```
1: string input = "abc";  
2: int number = int.Parse(input);  
3: Console.WriteLine(number);
```

正解

発生行：2行目

理由："abc"は数値として解釈できる形式ではない
ため、int.ParseがFormatExceptionを起こす

改善行：1行目

書き直し：string input = "123"; (数値文字列にする)

解説

int.Parse は文字列を整数に変換する関数だが、変換できない文字列を渡すと実行時に例外（FormatException）が発生する。コンパイル時には文字列の中身までは分からないため、コンパイルは通ってしまう点がポイント。

大問3-3

配点 6点

```
1: int[] data = { 1, 2, 3, 4, 5 };  
2: int index = data.Length - 5;  
3: Console.WriteLine(data[index - 1]);
```

正解

発生行：3行目

理由：data.Length(5)-5=0であるindexから1を引くと
-1になり、負の添字でアクセスしているため

改善行：3行目

書き直し：Console.WriteLine(data[index]);

解説

index の値を実際に計算すると0になる。そこからさらに1を引いた -1 という負の添字にアクセスしようとして `IndexOutOfRangeException` が発生する。

大問3-4

配点 6点

```
1: static int GetValue(int[] arr, int idx)
2: {
3:     return arr[idx];
4: }
5:
6: static void Main(string[] args)
7: {
8:     int[] nums = { 10, 20, 30 };
9:     int result = GetValue(nums, 3);
10:    Console.WriteLine(result);
11: }
```

正解

発生行：3行目

理由：9行目でidx=3を渡しているが、numsの有効な
添字は0～2（要素数3）であるため

改善行：9行目

書き直し：int result = GetValue(nums, 2); (例)

解説

エラーが実際に発生する場所（3行目）と、その原因を作っている場所（9行目の呼び出し）が違うという、少し発展的な問題。「どこで止まるか」と「なぜ止まるか」を切り分けて考える力を見る。

4

プログラムの穴埋め

配点24点（各8点）

大問4-1

配点 8点

```
static int GetMax(int[] numbers)
{
    int max = numbers[0];
    for (int i = ① ; i < numbers.Length; i++)
    {
        if (numbers[i] > ② )
        {
            max = ③ ;
        }
    }
    return max;
}
```

正解

① 1 (3点) ② max (3点)

③ numbers[i] (2点)

解説

配列の先頭(numbers[0])を仮の最大値として、2番目の要素(index=1)から順に比較していく典型的な「最大値探索」のロジック。より大きい値が見つかるたびに max を更新する。

大問4-2

配点 8点

```
static string FizzBuzz(int n)
{
    if ( ① )
    {
        return "FizzBuzz";
    }
    else if ( ② )
    {
        return "Fizz";
    }
    else if ( ③ )
    {
        return "Buzz";
    }
    else
    {
        return n.ToString();
    }
}
```

正解

① `n % 3 == 0 && n % 5 == 0` (4点)

② `n % 3 == 0` (2点)

③ `n % 5 == 0` (2点)

解説

定番のFizzBuzz問題。今回はreturnの中身ではなく、if文の条件式そのものを書かせる形式。①は「3でも5でも割り切れる」という複合条件（&&）を、②③はそれぞれ単独の条件を正しく書けるかを見る。

大問4-3

配点 8点

```
static int CountEven(int[] arr)
{
    int count = ① ;
    for (int i = 0; i < arr.Length; i++)
    {
        if (arr[i] % 2 == ② )
        {
            count++;
        }
    }
    return count;
}

static void Main(string[] args)
{
    int[] data = { 3, 8, 15, 20, 7 };
    int result = ③ ;
    Console.WriteLine(result);
}
```

正解

- ① 0 (2点) ② 0 (3点)
③ CountEven(data) (3点)

解説

偶数の個数を数える集計処理。①はカウンタの初期値、②は「2で割った余りが0＝偶数」の判定、③は関数の呼び出し方（配列を引数として渡す）を問う。参考：data={3, 8, 15, 20, 7}の偶数は8と20で、実行結果は2。

5

コメント作成

配点16点

大問5

配点 16点

Judge / CountPass 関数

```
// ①
static string Judge(int score)
{
    if (score >= 60) { return "合格"; }
    else { return "不合格"; }
}

// ②
static int CountPass(int[] scores)
{
    int count = 0;
    for (int i = 0; i < scores.Length; i++)
    {
        // ③
        if (Judge(scores[i]) == "合格")
        {
            count++;
        }
    }
    return count;
}
```

Main 関数

```
static void Main(string[] args)
{
    int[] scores = { 45, 78, 60, 92, 33 };
    int sum = 0;

    // ④
    for (int i = 0; i < scores.Length; i++)
    {
        sum += scores[i];
        string result = Judge(scores[i]);
        Console.WriteLine((i+1) + "番目: " +
            scores[i] + "点 -> " + result);
    }

    // ⑤
    double average = (double)sum / scores.Length;
    Console.WriteLine("平均点: " + average);

    // ⑥
    int passCount = CountPass(scores);
    Console.WriteLine("合格者数: " + passCount + "人");
}
```

① (3点)

得点(score)を受け取り、60点以上なら”合格”、未満なら”不合格”を返す関数

② (3点)

配列の各要素をJudge関数で判定し、”合格”だった人数を数えて返す関数

③ (2点)

Judge(scores[i])が”合格”のとき、countを1増やしている（合格者を数えている）

④ (3点)

各生徒の点数を合計に加算しながら、Judge関数で合否を判定して1件ずつ表示するループ

⑤ (3点)

合計点を人数で割って平均点を求め、表示している（doubleにキャストして整数同士の割り算を避けている）

⑥ (2点)

CountPass関数を呼び出して合格者数を求め、表示している

採点・クラス分けの目安（参考）

以下はあくまで一案です。実際の人数分布や指導方針に応じて調整してください。

80～100点

応用クラス

コードを正確に読み、エラーの原因も自力で説明できるレベル。発展的な内容（クラス・LINQ等）に進んでよい。

50～79点

標準クラス

基本文法は理解できているが、実行時の挙動や型の扱いに不安が残る。演習量を増やしながら基礎を固める。

0～49点

基礎クラス

変数・条件分岐・繰り返しの読み取りから丁寧に復習する必要がある。トレース練習を重点的に行う。